



第三章 SQL及常规操作

华为云 + 智能, 见未来

www.huaweicloud.com



前言

- GaussDB(for openGauss)是一款高性能、高可用的关系型云数据库，在支持SQL标准的同时，还兼容主流商业数据库常用的特有SQL语法和能力。
- 本章主要讲述GaussDB(for openGauss)云数据库支持的数据类型、数据库对象、系统函数及操作符等内容的介绍，帮助初学者掌握SQL入门级的基础语法。



课程目标

- 学完本课程后，您将能够：
 - 熟悉常用的数据类型
 - 掌握主要的数据库对象
 - 熟悉常用的函数与操作符
 - 掌握数据查询与操作



目录

1. 数据类型
2. 常用数据库对象
3. 函数与操作符
4. 数据查询
5. 数据操纵

数据类型

- 和所有的计算机语言一样，SQL语言也有自己的数据类型，这些数据类型主要用于在创建基本表（关系）的时候指定基本表的每个列（属性）的类型。这些数据类型主要可以分成数值类型、字符串类型、时间/日期类型等。
 - 数值类型：数值类型又可以分成精确数值类型和近似数值类型，如int、bigint、float等。
 - 字符类型：主要可分为定长字符类型和变长字符类型如char(n)、varchar(n)。
 - 日期类型：主要包括日期和时间类型，如DATE和TIMESTAMP。
 - 其他类型：如二进制类型、布尔类型等。

- 数据类型是数据的一个基本属性，用于区分不同类型的数据。不同的数据类型所占的存储空间不同，能够进行的操作也不相同。数据库中的数据存储于数据表中。数据表中的每一列都定义了数据类型，用户存储数据时，须遵从这些数据类型的属性，否则可能会出错。
- 注意：在使用GaussDB(for openGauss)创建数据库时，会根据客户需求指定兼容的数据库类型。GaussDB(for openGauss)支持兼容的数据库类型有：MySQL、Teradata、Oracle、PostgreSQL。选择不同兼容数据库类型后，所使用到的数据类型有一定差异。因此用户需要在使用时注意数据类型的大小、兼容性等。

数值类型 (1)

- 整型类型

- TINYINT:

- 微整数，占用1字节，别名为INT1；
 - 取值范围：0~255。

- SMALLINT:

- 小范围整数，占用2字节，别名为INT2；
 - 取值范围：-32,768 ~ +32,767。

- INTEGER/BINARY_INTEGER:

- 常用整数，占用4字节，别名为INT4。

- BIGINT:

- 大范围整数，占用8字节，别名为INT8。
 - 取值范围：-9,223,372,036,854,775,808 ~ +9,223,372,036,854,775,807

- TINYINT、SMALLINT、INTEGER和BIGINT类型存储各种范围的数字，也就是整数。试图存储超出范围以外的数值将会导致错误。
- 常用的类型是INTEGER，因为它提供了在范围、存储空间、性能之间的最佳平衡。一般只有取值范围确定不超过SMALLINT的情况下，才会使用SMALLINT类型。而只有在INTEGER的范围不够的时候才使用BIGINT，因为前者相对快得多。

数值类型 (2)

- 任意精度型

- NUMERIC[(p[,s])]/DECIMAL[(p[,s])]:

- 用户声明精度。每四位（十进制位）占用两个字节，然后在整个数据上加上八个字节的额外开销。
 - p为总位数，s为小数位数。精度p取值范围为[1,1000]，标度s取值范围为[0,p]。
 - 取值范围：未指定精度的情况下，小数点前最大131,072位，小数点后最大16,383位。

- NUMBER[(p[,s])]:

- NUMERIC[(p[,s])]的别名

- 与整数类型相比，任意精度类型需要更大的存储空间，其存储效率、运算效率以及压缩比效果都要差一些。在进行数值类型定义时，优先选择整数类型。当且仅当数值超出整数可表示最大范围时，再选用任意精度类型。
- 使用Numeric/Decimal进行列定义时，建议指定该列的精度p以及标度s。

数值类型 (3)

- 单精度浮点数
 - REAL/FLOAT4
- 双精度浮点数
 - FLOAT8/DOUBLE PRECISION/BINARY_DOUBLE
- 其他
 - FLOAT[(p)]/DEC[(p[,s])]/INTEGER[(p[,s])]

- 关于浮点类型的精度，目前只能保证直接读取时的精度位数。涉及分布式计算时，由于计算执行在各个DN节点上，并且最终汇聚到一个CN节点，因此误差可能会随计算节点数量增加而被放大。

数值类型 (4)

- 序列整型
 - SMALLSERIAL
 - 2字节序列整型
 - SERIAL
 - 4字节序列整型
 - BIGSERIAL
 - 8字节序列整型

- SMALLSERIAL，SERIAL和BIGSERIAL类型不是真正的类型，只是为在表中设置唯一标识做的概念上的便利。因此，创建一个整数字段，并且把它的缺省数值安排为从一个序列发生器读取。应用了一个NOT NULL约束以确保NULL不会被插入。在大多数情况下用户可能还希望附加一个UNIQUE或PRIMARY KEY约束避免意外地插入重复的数值，但这个不是自动的。最后，将序列发生器从属于那个字段，这样当该字段或表被删除的时候也一并删除它。目前只支持在创建表时候指定SERIAL列，不可以在已有的表中，增加SERIAL列。另外临时表也不支持创建SERIAL列。因为SERIAL不是真正的类型，也不可以将表中存在的列类型转化为SERIAL。

字符类型(1)

- 定长字符串类型

- CHAR(n)
- CHARACTER(n)
- NCHAR(n)

- 类型说明：

- 定长字符串，不足补空格。n是指字节长度，如不带精度n，默认精度为1。
- 存储空间最大为10MB。

- 除了每列的大小限制以外，每个元组的总大小也不可超过1GB-8203字节（即1073733621字节）。
- 在GaussDB Kernel里另外还有两种定长字符类型：name类型只用在内部系统表中，作为存储标识符，不建议普通用户使用。该类型长度当前定为64字节（63可用字符加结束符）。类型"char"只用了一个字节的存储空间。他在系统内部主要用于系统表，主要作为简单化的枚举类型使用。

字符类型(2)

- 变长字符串类型

- CLOB/TEXT

- 存储文本大对象。
 - 最大为1GB-8203字节（即1073733621字节）。

- VARCHAR(n)

- 存储变长字符串，n最大为10M。
 - 对应关键字为VARCHAR/VARCHAR2/NVARCHAR2/CHARACTER VARYING。

- 对于存储内容知道指定长度的，如存储性别时，只有可能是男或者女，这时候采用 nchar(1)，可以节省存储空间。
- 对于存储内容长度不定的，可采用变长字符串类型varchar等，可以节省存储空间，因为不需要利用空格补齐。
- clob常用于存储长的数据内容，如对某产品的详细描述等。

日期类型 (1)

名称	描述	存储空间
DATE	日期。	4字节
TIME [(p)] [WITHOUT TIME ZONE]	只用于一日内时间。 p表示小数点后的精度，取值范围为0~6。	8字节
TIME [(p)] [WITH TIME ZONE]	只用于一日内时间，带时区。 p表示小数点后的精度，取值范围为0~6。	12字节
TIMESTAMP[(p)] [WITHOUT TIME ZONE]	日期和时间。 p表示小数点后的精度，取值范围为0~6。	8字节
TIMESTAMP[(p)][WITH TIME ZONE]	日期和时间，带时区。TIMESTAMP的别名为TIMESTAMPPTZ。 p表示小数点后的精度，取值范围为0~6。	8字节
SMALLDATETIME	日期和时间，不带时区。 精确到分钟，秒位大于等于30秒进一位。	8字节

- 1. 时间类型的数据在显示的时候会自动忽略末尾的所有零。
- 2. 精度p默认取值为6。
- 3. 对于INTERVAL类型，日期和时间在系统内部分别用int32和double类型存储，所以两者的取值范围 and 对应数据类型的取值范围一致。
- 4. 插入时间超出范围的时候，系统可能不报错，但不保证行为正常。

日期类型 (2)

名称	描述	存储空间
INTERVAL DAY (l) TO SECOND (p)	时间间隔，X天X小时X分X秒。 l: 天数的精度，取值范围为0~6。为适配Oracle语法，未实现具体功能。 p: 秒数的精度，取值范围为0~6。小数末尾的零不显示。	16字节
INTERVAL [FIELDS] [(p)]	时间间隔。 fields: 可以是YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, DAY TO HOUR, DAY TO MINUTE, DAY TO SECOND, HOUR TO MINUTE, HOUR TO SECOND, MINUTE TO SECOND等 p: 秒数的精度，取值范围为0~6，且fields为SECOND, DAY TO SECOND, HOUR TO SECOND或MINUTE TO SECOND时，参数p才有效。 小数末尾的零不显示。	12字节
reltime	相对时间间隔。格式为： X years X mons X days XX:XX:XX。 采用儒略历计时，规定一年为365.25天，一个月为30天，计算输入值对应的相对时间间隔，输出采用POSTGRES格式。	4字节
abstime	日期和时间。格式为： YYYY-MM-DD hh:mm:ss+timezone 取值范围为1901-12-13 20:45:53 GMT~2038-01-19 03:14:04 GMT，精度为秒。	4字节

- 1. 时间类型的数据在显示的时候会自动忽略末尾的所有零。
- 2. 精度p默认取值为6。
- 3. 对于INTERVAL类型，日期和时间在系统内部分别用int32和double类型存储，所以两者的取值范围和对数据类型的取值范围一致。
- 4. 插入时间超出范围的时候，系统可能不报错，但不保证行为正常。

二进制类型

名称	描述	存储空间
BLOB	二进制大对象 说明（列存不支持BLOB类型）	1~8000最大为1GB-8203字节（即1073733621字节）
RAW	变长的十六进制类型 说明（列存不支持RAW类型）	4字节加上实际的十六进制字符串。最大为1GB-8203字节（即1073733621字节）。
BYTEA	变长的二进制字符串	4字节加上实际的二进制字符串。最大为1GB-8203字节（即1073733621字节）。

- 除了每列的大小限制以外，每个元组的总大小也不可超过1GB-53字节（即1073741771字节）。
- 不支持直接使用BYTEAWITHOUTORDERWITHEQUALCOL和BYTEAWITHOUTORDERCOL，_BYTEAWITHOUTORDERWITHEQUALCOL，_BYTEAWITHOUTORDERCOL类型创建表。

其他数据类型

- 布尔类型
- 货币类型
- 几何类型
- UUID类型
- JSON类型
- 网络地址类型

- GaussDB Kernel支持某些数据类型间的隐式转换。
- 布尔类型可与int和bigint类型转换，因为布尔类型可以看成是数字0与1，因此它可以转换为整数0和1，整数类型可以转换为布尔类型，转换规则为整数0对应布尔false，其他非0整数为布尔true。
- n的取值范围是[0,4]，表示年的精度，默认值为2。
- n1取值范围是[0,7]，表示天的精度，默认值为2。
- n2取值范围是[0,6]，表示秒后面的精度，不指定时默认为6。

思考题

1. (判断题) blob用于存储变长大对象二进制数据? ()

- A. True
- B. False

· 参考答案: A.True

思考题

1. (多选题)以下哪些是时间间隔类型? ()

- A. timestamp[(n)]
- B. timestamp(n) with time zone
- C. interval day[(n1)] to second [(n2)]
- D. interval year[(n)] to month

- 参考答案: CD。A、B是时间点,非时间间隔类型。

目录

1. 数据类型
- 2. 常用数据库对象**
3. 函数与操作符
4. 数据查询
5. 数据操纵

数据库对象 (1)

- 什么是数据库对象？
 - 数据库对象是数据库的组成部分，数据库对象主要包含：表，索引，视图，存储过程，触发器，用户，函数等。
- 表
 - 表是数据库中的一种特殊数据结构，用于存储数据对象以及对象之间的关系，由行和列组成的。
- 索引
 - 索引是对数据库表中一列或多列的值进行排序的一种结构，使用索引可快速访问数据库表中的特定信息。
- 视图
 - 视图是从一个或几个基本表中导出的虚表，可用于控制用户对数据访问。
- 存储过程
 - 存储过程是一组为了完成特定功能的SQL语句的集合。一般用于报表统计、数据迁移等。

数据库对象 (2)

- 触发器
 - 触发器是一种特殊类型的存储过程，通过指定的事件触发执行。一般用于数据审计、数据备份等。
- 函数
 - 函数是对一些业务逻辑的封装，以完成特定的功能。函数执行完成后会返回执行结果。

表

- 表是数据库中的一种特殊数据结构，用于存储数据对象以及对象之间的关系。所涉及的SQL语句如下表所示。

功能	相关SQL
创建表	CREATE TABLE
修改表属性	ALTER TABLE
删除表	DROP TABLE
删除表中所有数据	TRUNCATE TABLE

创建表语法

- 语法格式:

```
CREATE [ [ GLOBAL | LOCAL ] [ TEMPORARY | TEMP ] | UNLOGGED ] TABLE [ IF NOT EXISTS ]
table_name
({ column_name data_type [ compress_mode ] [ COLLATE collation ] [ column_constraint [ ... ] ]
| table_constraint
| LIKE source_table [ like_option [ ... ] ] }
[ ... ]
[ WITH ( { storage_parameter = value } [ ... ] ) ]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS } ]
[ COMPRESS | NOCOMPRESS ]
[ TABLESPACE tablespace_name ]
-- 以下为分布式独有
[ DISTRIBUTE BY { REPLICATION | { HASH ( column_name [ ... ] )
| { RANGE ( column_name [ ... ] ) SLICE REFERENCES tablename | ( slice_less_than_item [ ... ]
| slice_start_end_item [ ... ] )
| { LIST ( column_name [ ... ] ) SLICE REFERENCES tablename | ( slice_values_item [ ... ]
)}} } ]
[ TO { GROUP groupname | NODE ( nodename [ ... ] ) } ];
```

- 注意:

- 创建列存的数量建议不超过1000个。
- 分布式版中表的主键约束和唯一约束必须包含分布列。

- 其他注意事项:
- 行存表的表级约束不支持外键。
- 列存表的字段约束只支持NULL、NOT NULL和DEFAULT常量值。
- 如果在建表过程中数据库系统发生故障，系统恢复后可能无法自动清除之前已创建的、大小为0的磁盘文件。此种情况出现概率小，不影响数据库系统的正常运行。

主要参数介绍

- 创建表的参数说明

- GLOBAL：创建全局表
- TEMPORARY：创建临时表
- IF NOT EXISTS：创建表时，如果表已经存在，则不做改动直接返回；如果表不存在，则创建新表
- table_name：要创建的表名
- column_name：要创建表的列名
- column_constraint：列约束
- DISTRIBUTE BY { REPLICATION | { HASH (column_name [,...]) }：指定表的分布键

- 临时表：临时表只在当前会话可见，本会话结束后将自动删除。一般用于存放中间数据。
- 分区表：通常会将大表按照某个字段进行划分，物理划分为多个分区表。在访问或操作该表中的数据时，减小扫描数据量，提升效率。
- 分布式数据库GaussDB(for openGauss)中表的分布
 - REPLICATION：表的每一行存在所有数据节点（DN）中，即每个数据节点都有完整的表数据。
 - HASH (column_name)：对指定的列进行Hash，通过映射，把数据分布到指定DN。
 - RANGE(column_name) 对指定列按照范围进行映射，把数据分布到对应DN。
 - LIST(column_name) 对指定列按照具体值进行映射，把数据分布到对应DN。

创建表示例

- 示例:

- 创建表education

```
CREATE TABLE education(staff_id INT, highest_degree CHAR(8) NOT NULL, graduate_school  
VARCHAR(64), graduate_date DATETIME, education_note VARCHAR(70));
```

- 创建分区表training

```
CREATE TABLE training(staff_id INT NOT NULL, course_name CHAR(20), course_period DATETIME,  
exam_date DATETIME, score INT)  
PARTITION BY RANGE(staff_id)  
(  
PARTITION training1 VALUES LESS THAN(100),  
PARTITION training2 VALUES LESS THAN(200),  
PARTITION training3 VALUES LESS THAN(300),  
PARTITION training4 VALUES LESS THAN(MAXVALUE)  
);
```

修改表属性

- ALTER TABLE功能指通过更改、添加、删除列和约束来更改表的定义，功能包括：
 - 列的添加、删除、修改、重命名
 - 约束的添加、删除
 - 约束的启动和禁用
 - 修改分区的名称
- 语法格式

```
ALTER TABLE [ schema_name. ]table_name
{ alter_table_properties
| column_clauses
| references_clause
| constraint_clauses
| partition_clauses
| enable_disable_clause
| set_interval_clause
}
```

修改表示例

- 示例：

- 添加列full_masks。

```
ALTER TABLE training ADD full_masks INT;
```

- 删除列course_period。

```
ALTER TABLE training DROP course_period;
```

- 修改列的数据类型。

```
ALTER TABLE training MODIFY course_name VARCHAR(20);
```

- 添加约束。

```
ALTER TABLE training ADD CONSTRAINT ck_training CHECK(staff_id>0);  
ALTER TABLE training ADD CONSTRAINT uk_training UNIQUE(course_name);
```

删除表

- 注意

- DROP TABLE会强制删除指定的表，删除表后，依赖该表的索引会被删除，而使用到该表的函数和存储过程将无法执行。删除分区表，会同时删除分区表中的所有分区。
- 只有表的所有者或者被授予了表的DROP权限的用户才能执行DROP TABLE，系统管理员拥有该权限。

- 语法格式

```
DROP TABLE [ IF EXISTS ] { [ schema. ] table_name } [, ...] [ CASCADE | RESTRICT ];
```

- 参数说明

- CASCADE CONSTRAINTS
- 级联删除依赖于表的对象（比如视图）。

- 示例

```
DROP TABLE IF EXISTS training;
```

索引

- 索引是对数据库表中一列或多列的值进行排序的一种结构，使用索引可快速访问数据库表中的特定信息。所涉及的SQL语句，如下表所示。

功能	相关SQL
创建索引	CREATE INDEX
修改索引属性	ALTER INDEX
删除索引	DROP INDEX

- 索引按照索引列数分为单列索引和多列索引，按照索引使用方法可以分为普通索引、唯一索引、函数索引、分区索引。

索引类型简介

- 按照索引列：
 - 单列索引：即一个索引只包含表中的单个列(字段)。
 - 多列索引：即一个索引包含表中的多个列(字段)。
- 按照索引使用方法：
 - 普通索引：最基本的索引，没有其他限制条件。
 - 唯一索引：与普通索引类似，但其索引列的值必须是唯一的，否则创建报错。
 - 函数索引：基于对表中列进行计算后的结果创建索引。
 - 分区索引：针对分区表中的分区创建的索引。

- 注意：唯一索引允许有空值(NULL)。

创建索引

- 功能描述

- 在指定的表上创建一个索引。索引可以用来提高数据库查询性能，但是不恰当的使用将导致性能下降。

- 注意

- 索引自身也占用存储空间、消耗计算资源，创建过多的索引将对数据库性能造成负面影响。因此，仅在必要时创建索引。
- 在分区表上创建唯一索引时，索引项中必须包含分布列和所有分区键。

- 语法格式

```
CREATE [ UNIQUE ] INDEX [ [schemaname.]index_name ] ON table_name [ USING method ]  
( { { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [ ASC | DESC ] [ NULLS  
{ FIRST | LAST } } ] [, ... ] )  
[ WITH ( {storage_parameter = value} [, ... ] ) ]  
[ TABLESPACE tablespace_name ]  
[ WHERE predicate ];
```

主要参数介绍

- UNIQUE
 - 创建唯一性索引，每次添加数据时检测表中是否有重复值。
- index_name
 - 要创建的索引名。
- table_name
 - 要创建索引的表名，可以有用户修饰。
- column_name
 - 表中需要创建索引列的名称（字段名）。
- expression
 - 创建一个基于该表的一个或多个字段的表达式索引。
- PARTITION index_partition_name
 - 指定分区索引

创建索引示例

- 在普通表posts上创建索引。

```
--创建普通表posts。
CREATE TABLE posts(post_id CHAR(2) NOT NULL, post_name CHAR(6) PRIMARY KEY, basic_wage INT,
basic_bonus INT);
--创建索引idx_posts。
CREATE INDEX idx_posts ON posts(post_id ASC, post_name);
```

- 在分区表education上创建分区索引。

```
--创建分区表education。
CREATE TABLE education(staff_id INT NOT NULL, highest_degree CHAR(8), graduate_school VARCHAR(64),
graduate_date DATETIME, education_note VARCHAR(70))
PARTITION BY LIST(highest_degree)
(
PARTITION doctor VALUES ('博士'),
PARTITION master VALUES ('硕士'),
PARTITION undergraduate VALUES ('学士')
);
--创建分区索引。
CREATE INDEX idx_training ON education(staff_id ASC, highest_degree) LOCAL (PARTITION doctor, PARTITION master,
PARTITION undergraduate);
--创建函数索引。
CREATE INDEX idx_func ON education(upper(graduate_school));
```

修改索引属性

- 语法格式：

```
ALTER INDEX [ schema_name. ] index_name [ ON [ schema_name. ] table_name ]  
{ rebuild_clauses  
| rename_clauses  
}
```

- rebuild_clauses

- REBUILD [PARTITION index_partition_name]
- 重建表索引或索引分区。

- rename_clauses

- RENAME TO [schema_name.] index_name_new
- 待重命名的索引名。

修改索引示例

- 创建索引idx_posts。

```
CREATE INDEX idx_posts ON posts(post_id ASC, post_name);
```

- 在线重建索引。

```
ALTER INDEX idx_posts REBUILD;
```

- 重命名索引。

```
ALTER INDEX idx_posts RENAME TO idx_posts_temp;
```

删除索引

- 语法格式

```
DROP INDEX [ CONCURRENTLY ] [ IF EXISTS ]  
index_name [, ...] [ CASCADE | RESTRICT ];
```

- 参数说明

- IF EXISTS: 索引不存在时, 直接返回成功。
- index_name: 待删除索引名。
- CASCADE | RESTRICT: CASCADE表示允许级联删除依赖于该索引的对象, RESTRICT表示有依赖于此索引的对象存在则索引无法被删除。

- 示例

```
DROP INDEX IF EXISTS idx_posts ON posts;
```

视图

- 视图是从一个或几个基本表中导出的虚表，可用于控制用户对数据访问，所涉及的SQL语句，如下表所示。

功能	相关SQL
创建视图	CREATE VIEW
修改视图	ALTER VIEW
删除视图	DROP VIEW

- 视图与基本表不同，数据库中仅存放视图的定义，而不存放视图对应的数据，这些数据仍存放在原来的基本表中。若基本表中的数据发生变化，从视图中查询出的数据也随之改变。从这个意义上讲，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据及变化。

- 针对视图的语法支持ALTER VIEW去修改视图的属性。但实际工作当中很少用到，更多的是使用CREATE OR REPLACE进行直接替换。

创建视图

- 语法格式

```
CREATE [ OR REPLACE ] VIEW [ schema_name. ]view_name [ ( alias [ ,... ] ) ] AS subquery
```

- 参数说明

- OR REPLACE: 创建视图时, 若视图存在则更新。
- schema_name.view_name: 视图名。
- alias: 视图列别名, 若不给出, 将根据后面子查询自动推导列名。
- AS subquery: 子查询。

创建视图示例

- 创建视图privilege_view，若该视图存在则更新该视图。

```
CREATE OR REPLACE VIEW training_view AS SELECT staff_id, score from training;
```

- 创建视图privilege_view并指定视图列别名，若该视图存在则更新该视图。

```
CREATE OR REPLACE VIEW training_view(id,course,date,score) AS SELECT * FROM training;
```

- 查看视图中的数据，语法和查询表一样。

```
SELECT * FROM training_view;
```

- 查看视图结构。

```
\d+ training_view;
```

删除视图

- 语法格式

```
DROP VIEW [ IF EXISTS ] [ schema_name. ]view_name
```

- 参数说明

- IF EXISTS: 视图存在, 则执行删除。
- schema_name.view_name: 待删除的视图。

- 示例

```
DROP VIEW IF EXISTS privilege_view;
```

其他对象

- 存储过程

- 商业规则和业务逻辑可以通过程序存储在GaussDB(for openGauss)中，这个程序就是存储过程。存储过程是SQL、PL/SQL、Java语句的组合。存储过程使执行商业规则的代码可以从应用程序中移动到数据库。从而，代码存储一次能够被多个程序使用。

- 触发器

- 可以实现对数据修改事件的捕获，并在之后触发用户自定义的操作行为。触发器的触发时机是可设置的，可以是语句级触发或者记录级触发，可以是修改前触发也可以是修改后触发。

- 约束

- 定义规则使得数据能够朝确定的方向发展，也就是保证数据的完整性。主要的约束有主键约束、非空约束、唯一约束等。

思考题

1. （多选题） GaussDB(for openGauss)支持哪些哪些索引（ ）

- A. 唯一索引
- B. 函数索引
- C. 多列索引
- D. 单列索引

· 参考答案： ABCD

目录

1. 数据类型
2. 常用数据库对象
- 3. 函数与操作符**
4. 数据查询
5. 数据操纵

系统函数

- 系统函数是对一些业务逻辑的封装，以完成特定的功能。系统函数可以有参数，也可以没有参数。系统函数执行完成后会返回执行结果。
- 系统函数的分类如下：
 - 数值计算函数
 - 字符处理函数
 - 时间日期函数
 - 间隔函数
 - 类型转换函数

- 系统函数的主要作用是简化应用开发人员的很多工作，减少数据在数据库和应用服务器之间的传输，提高数据处理的效率。
- 例如可以直接使用replace函数对数据库中的字符串进行替换，而不需要写其他的代码去实现这个功能。

数值计算函数 (1)

- `abs(exp)`, `cos(exp)`, `sin(exp)`, `acos(exp)`, `asin(exp)`: 返回表达式的绝对值, 余弦值, 正弦值, 反余弦值和反正弦值。
- `abs(exp)`的返回值类型与参数`exp`数据类型相同, 其他的返回值为`number`。
- `asin`和`acos`函数说明: 入参`exp`是可转成数值型的表达式, 取值范围为`[-1,1]`。

```
select abs(-10),cos(0),sin(0),acos(1),asin(0) from sys_dummy;
```

ABS(-10)	COS(0)	SIN(0)	ACOS(1)	ASIN(0)
10	1	0	0	0

1 rows fetched.

数值计算函数 (2)

- `bitand(exp1,exp2)`, `bitor(exp1,exp2)`, `bitxor(exp1,exp2)`: 按位与, 按位或, 按位异或运算。

```
select bitand(29,15),bitxor(29,15),bitor(29,15) from sys_dummy;
```

BITAND(29,15)	BITXOR(29,15)	BITOR(29,15)
13	18	31

1 rows fetched.

- `round(number[,decimals])`: 将`number`类数值按照`decimals`指定的向小数点前后截断。

```
select round(1234.5678,-2),round(1234.5678,2) from sys_dummy;
```

ROUND(1234.5678,-2)	ROUND(1234.5678,2)
1200	1234.57

1 rows fetched.

数值计算函数 (3)

语法	功能	示例
ceil(exp)	返回大于或者等于指定表达式n的最小整数。	ceil(15.3) → 16
rawtohex(exp)	将raw类数值exp转换为一个相应的十六进制表示的字符串。	rawtohex('012') → 303132
sign(exp)	取exp结果的符号，大于0返回1，小于0返回-1，等于0返回0。	sign(2*3) → 1
sort(n)	返回非负实数的平方根。入参是可转成非负数值型表达式。	sort(49) → 7
trunc(number, scale)	按指定的格式截取输入的数值数据。number是待截取的数据，scale截取精度。	trunc(15.79,1) → 15.7;trunc(15.79,-1) → 10
floor(exp)	求小于或等于表达式值的最近的整数。入参是可转成数值型的表达式。返回值是number。	floor(12.8) → 12
mod(exp1, exp2)	求模运算。入参是可转为number类型的表达式。返回值是number	mod(29,3) → 2
power(base, expn)	返回数字乘幂的计算结果。入参base表示底数，expn表示指数。返回值是number。	power(5,3) → 125

字符处理函数 (1)

- `concat(str[,...])`, `concat_ws(separator,str1,str2,...)`: 拼接一个或多个字符串。第一个函数无分隔符，第二个函数可以指定分隔符连接。

```
select concat('11','null','22'),concat_ws('-', '11',null,'22') from sys_dummy;  
  
CONCAT('11','NULL','22')    CONCAT_WS('-', '11',NULL,'22')  
-----  
11null22                    11-22
```

- `hextoraw(str)`: 将一个十六进制构成的字符串转换为二进制。

```
select hex('ABC'),hex2bin('0x28'),hextoraw('abc') from sys_dummy;  
  
HEXTORAW('ABC')  
-----  
0ABC
```

字符处理函数 (2)

- `replace(str,src,dst)`: 字符串插入和字符串替换函数。

```
select replace('123456','45','abds') from sys_dummy;  
REPLACE('123456','45','ABDS')  
-----  
123abds6
```

- `instr(str1,str2[,pos[,n]])`: 字符串查找函数。返回要查找的字符串在源字符串中的位置。返回在string1中从int1位置开始匹配到第int2次string2的位置，第一个int表示开始匹配起始位置，第二个int表示匹配的次数。

```
SELECT INSTR('abcdabcdabcd', 'bcd'), INSTR('abcdabcdabcd', 'bcd', 2, 2 );  
INSTR('abcdabcdabcd', 'bcd')    INSTR( 'abcdabcdabcd', 'bcd', 2, 2 )  
-----  
2                                6
```

字符处理函数 (3)

语法	功能	示例
left(str, length)	返回指定字符串的左边几位字符。。	left('abcdef',3) → abc left('abcdef',0) 或left('abcdef',0) → 空串
length(str)	获取字符串长度的函数。	length('1234大') → 5
lower(str)	将字符串转换成对应字符的小写。	lower('ABCD') → abcd lower('1234') → 1234
upper(str)	将字符串转换成对应字符的大写。	upper('abcd') → ABCD upper('1234') → 1234
right(str,len)	返回指定字符串的右边几位字符。	right('abcdef',3) → def right('abcdef',0) 或right('abcdef',0) → 空串
reverse(str)	返回字符串的倒序。仅支持string类型。	reverse('abcd') → dcba
substr(<i>str</i> , <i>start</i> [, <i>len</i>])	字符串截取函数。	substr('abcdefg',3,4) → cdef 表示从'abcdefg'字符串的第3个字符开始截取长度为4的字符。

时间日期函数 (1)

- `add_months(date,n)`：返回date加或减n个月后的值。

```
select add_months(to_date('2016-02-10','YYYY-MM-DD'),1);  
  
ADD_MONTHS(TO_DATE('2016-02-10','YYYY-MM-DD'),1)  
-----  
2016-03-10 00:00:00
```

- `extract(field from datetime)`：从指定的日期（datetime）中提取指定的时间字段（field），按指定的格式截取输入的日期数据。

```
select extract(month from '2019-08-23'::date);;  
  
extract(month from '2019-08-23'::date)  
-----  
8
```

时间日期函数 (2)

语法	功能	示例
current_timestamp	获取当前系统时间及时区。	SELECT current_timestamp; 2021-06-01 23:59:07.498876+08
now(fractional_second_precision)	获取当前系统时间及时区。	SELECT now(); 2021-06-01 23:59:07.498876+08
age(timestamp, timestamp)	描述：将两个参数相减，并以年、月、日作为返回值。若相减值为负，则函数返回亦为负。 返回值类型：interval	SELECT age(timestamp '2001-04-10', timestamp '1957-06-13'); 43 years 9 mons 27 days
current_date	描述：当前日期。 返回值类型：date	SELECT current_date; 2021-09-01
trunc(timestamp)	描述：默认按天截取。	SELECT trunc(timestamp '2001-02-16 20:38:40'); 2001-02-16 00:00:00
numtodsinterval(num, 'interval_unit')	输入一个数值和interval域描述字段，输出interval day to second类型。	SELECT sysdate; 2022-05-17 11:21:37 返回当前时间1小时后的时间： SELECT sysdate+numtodsinterval(1,'hour'); 2019-08-23 13:52:43

- 获取当前时间有多种方式，请根据实际业务从场景选择合适的接口：
（1）以下接口按照当前事务的开始时刻返回值：
CURRENT_DATE CURRENT_TIME CURRENT_TIMESTAMP CURRENT_TIME(precision)
CURRENT_TIMESTAMP(precision) LOCALTIME LOCALTIMESTAMP
LOCALTIME(precision) LOCALTIMESTAMP(precision)
其中CURRENT_TIME和CURRENT_TIMESTAMP传递带有时区的值；LOCALTIME和LOCALTIMESTAMP传递的值不带时区。CURRENT_TIME、CURRENT_TIMESTAMP、LOCALTIME和 LOCALTIMESTAMP可以有选择地接受一个精度参数，该精度导致结果的秒域被园整为指定小数位。如果没有精度参数，结果将被给予所能得到的全部精度。
因为这些函数全部都按照当前事务的开始时刻返回结果，所以它们的值在事务运行的整个期间内都不改变。我们认为这是一个特性：目的是为了允许一个事务在“当前”时间上有一致的概念，这样在同一个事务里的多个修改可以保持同样的时间戳。
（2）以下接口返回当前语句开始时间：
transaction_timestamp() statement_timestamp() now()
其中transaction_timestamp()等价于CURRENT_TIMESTAMP，但是其命名清楚地反映了它的返回值。statement_timestamp()返回当前语句的开始时刻（更准确的说是收到 客户端最后一条命令的时间）。statement_timestamp()和transaction_timestamp()在一个事务的第一条命令期间返回值相同，但是在随后的命令中却不一定相同。
now()等效于transaction_timestamp()。
（3）以下接口返回函数被调用时的真实当前时间：
clock_timestamp() timeofday()

`clock_timestamp()`返回真正的当前时间，因此它的值甚至在同一条 SQL 命令中都会变化。`timeofday()`和`clock_timestamp()`相似，`timeofday()`也返回真实的当前时间，但是它的结果是一个格式化的text串，而不是timestamp with time zone值。

条件表达式函数

- `coalesce(expr1, expr2, ..., exprn)`: 返回参数列表中第一个非NULL的参数值。
- `nullif(expr1, expr2)`: 当且仅当`expr1`和`expr2`相等时, NULLIF才返回NULL, 否则它返回`expr1`。
- `nvl( expr1 , expr2 )`: 如果`expr1`为NULL则返回`expr2`; 如果`expr1`非NULL, 则返回`expr1`。

```
SELECT coalesce(NULL, 'hello');
coalesce
-----
hello
(1 row)

SELECT nullif(10,12), nvl(null,12);
nullif | nvl
-----+-----
10     | 12
(1 row)
```

`decode(base_expr, compare1, value1, Compare2,value2, ... default)`描述: 把`base_expr`与后面的每个`compare(n)` 进行比较, 如果匹配返回相应的`value(n)`。如果没有发生匹配, 则返回`default`。

类型转换函数 (1)

- to_char (datetime/interval [, fmt]), 转换为字符串
- to_clob(char/nchar/varchar/nvarchar/varchar2/nvarchar2/text/raw), 转换为CLOB
- to_date(text), to_number(text, text): 将字符串类型的值转换为指定格式的数字。

```
select to_char(35),to_clob('excellent staff of 2018'),to_date('2019-08-23','YYYY-MM-DD'),to_number('12E','xxxx');
to_char | to_clob | to_date | to_number
-----+-----+-----+-----
35      | excellent staff of 2018 | 2019-08-23 | 302
(1 row)
```

类型转换函数 (2)

语法	功能	示例
ascii(str)	返回字符串str首个字符对应的ASCII码。	ascii('hello') → 104
cast(expr as datatype)	将列名/值转换为指定的数据类型datatype。表达式可以转换为与自身相同的类型。	cast('10' as int) → 10
chr(n)	返回ascii码为n的字符。n支持范围为[0,127]。入参是可转成数值型表达式。	chr(67) → C
convert(expr, data_type)	将expr转换成data_type类型。data_type取值范围是除了clob, blob, image以外的所有数据类型。	convert('2018-06-28 13:14:15', timestamp) → 2018-06-28 13:14:15.000000
to_hex(number int or bigint)	把number转换成十六进制表现形式。	to_hex(2147483647) → 7fffffff

- openGauss=# select pg_catalog.char(67);
- char
- -----
- C
- (1 row)

操作符

- 操作符可对一个或多个操作数进行处理，位置上可能处于操作数之前、之后，或两个操作数之间。
- 常见操作符类型（从使用场景划分）
 - 逻辑操作符
 - 比较操作符
 - 算术运算符
 - 通配符
 - 其他操作符

逻辑操作符

- 常用的逻辑操作符有AND、OR和NOT，他们的运算结果有三个值，分别为TRUE、FALSE和NULL，其中NULL代表未知。他们运算优先级顺序为：NOT>AND>OR。

a	b	a AND b	a OR b	NOT a
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL

- 重点关注NULL值和其他值逻辑运算后的结果。

比较操作符

操作符	描述
<	小于
>	大于
<=	小于或等于
>=	大于或等于
=	等于
<> 或 !=	不等于

```
--从staffs表中查询薪酬>5000的职员信息。  
select * from staffs where salary>5000;  
--从staffs表中查询薪酬不等于5000的职员信息。  
select * from staffs where salary<>5000;
```

算数操作符

运算符	描述	运算符	描述
+	加		字符串拼接
-	减		按位或
*	乘	&	按位与
/	除（除法操作符不会取整）	#	按位异或
%	模运算	<<	左移位
		>>	右移位

```
select 2+3,2*3,'a' || 'b',3<<1 from sys_dummy;
```

2+3	2*3	'A' 'B'	3<<1

5	6	ab	6

```
1 rows fetched.
```

其他操作符

- 通配符

通配符	描述
%	表示任意数量的字符，包括无字符，用于like和not like语句中。
_	下划线，表示确切的一个未知字符，用于like和not like语句中。

- 其他操作符

操作符	描述
单引号 (')	表示字符串类型。如果在字符串文本里含有单引号，那么必须运用两个单引号示意。
双引号 (")	表示表、字段、索引等Object Name或者是别名

- 通配符实例

- select * from tab_users where username like 'Th%'; --表示在表中查询用户名为Th开头的用户信息。
- select * from tab_users where username like 'Th_mas'; --表示在表中查询用户名如 'Th_mas' 的用户信息，其中_可以代表任意一个字符（不是多个字符）。

目录

1. 数据类型
2. 常用数据库对象
3. 函数与操作符
- 4. 数据查询**
5. 数据操纵

数据查询

- 数据查询SQL顾名思义就是使用SQL语言对表中数据进行查询，主要介绍下常用的几种查询方式。
 - 条件查询
 - join连接查询
 - 子查询
 - 结果集查询
 - 聚集查询
 - 数据排序、限制查询

简单查询

- 日常查询中，最常用的是通过FROM子句实现的查询。
- 语法格式：

```
SELECT [ , ... ] FROM table_reference [ , ... ]
```

- 使用方法：
 - SELECT关键字之后和FROM子句之前出现的表达式称为SELECT项。SELECT项用于指定要查询的列，FROM指定要从哪个表中查询。如果要查询所有列，可以在SELECT后面使用*号，如果只查询特定的列，可以直接在SELECT后面指定列名，列名之间用逗号隔开。

- 数据查询语言是数据库最基本的应用，其语法较为复杂，包括简单查询、带条件查询、连接查询、子查询、集合运算、数据分组、排序和限制等，本节基于实际使用场景描述了数据查询语言的类型，以及如何使用它们。

简单查询示例

- 创建一个training表，并向表中插入三行数据后。查看training表的所有列。

```
--创建training表。  
CREATE TABLE training(staff_id INT NOT NULL,course_name CHAR(50), exam_date DATETIME,score INT);  
--向表中插入三行数据。  
INSERT INTO training(staff_id,course_name,exam_date,score)VALUES(10,'SQL majorization','2017-06-25 12:00:00',90);  
INSERT INTO training(staff_id,course_name,exam_date,score)VALUES(10,'information safety','2017-06-26 12:00:00',95);  
INSERT INTO training(staff_id,course_name,exam_date,score)VALUES(10,'master all kinds of thinking methonds','2017-07-25 12:00:00',97);
```

- SELECT后面使用*号查询training表中的所有列。

```
SELECT * FROM training;
```

STAFF_ID	COURSE_NAME	EXAM_DATE	SCORE
10	SQL majorization	2017-06-25 12:00:00	90
10	information safety	2017-06-26 12:00:00	95
10	master all kinds of thinking methonds	2017-07-25 12:00:00	97

• 此处随堂演示

去除重复值

- DISTINCT关键字
- 从SELECT的结果集中删除所有重复的行，使结果集中的每行都是唯一的。取值范围：已存在的字段名，或字段表达式。
- 语法格式
- 如果在DISTINCT关键字后只有一个列，则使用该列来计算重复，如果有两列或者多列，则将使用这些列的组合来进行重复检查。

```
SELECT DISTINCT [ , ... ] FROM table_reference [ , ... ]
```

去除重复值示例

- 下表中是一个部门的员工信息，利用distinct关键字来查询员工的岗位和奖金，去除岗位和奖金相同的记录。

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	tester	7000
34	denggui	tester	7000
35	caoming	developer	10000
37	lixue	developer	9000

```
select distinct job, bonus from sections;
```

JOB	BONUS
developer	9000
tester	7000
developer	10000

3 rows fetched.

- 通过distinct关键字，实现对job，bonus字段相同的记录去重。

查询列的选择 (1)

- 在选择查询列时，列名可以用下面几种形式表达：
- 手动输入列名，多个列之间用英文逗号(,)分隔。

```
SELECT a, b, f1, f2 FROM t1, t2;
```

- 其中，列a、b是表t1中的列，f1、f2是表t2中的列。
- 可以是计算出来的字段。

```
SELECT a+b FROM t1;
```

- 如果某两个或某几个表正好有一些共同的列名，推荐使用表名限定列名。不限定列名可以得到查询结果，但使用完全限定的表和列名称，可以减少数据库内部的处理工作量，从而提升查询的返回性能。例如：

```
SELECT t1.f1, t2.f1 from t1, t2;
```

查询列的选择 (2)

- 示例：查看training表中参与培训的员工编号及培训课程名。

```
SELECT staff_id, course_name FROM training;  
STAFF_ID    COURSE_NAME
```

```
-----  
10          SQL majorization  
10          information safety  
10          master all kinds of thinking methonds
```


查询列的选择 (3)

- 示例：表名限定列名，查询学号sid为10的学生的数学成绩和英语成绩。

- 数学成绩表math

sid	score
10	95
11	87
12	99

- 英语成绩表english

sid	score
10	82
11	87
12	93

```
select a.sid,a.score as math, b.score as english from math a,english b where a.sid = 10 and b.sid = 10;
```

```
SID      MATH      ENGLISH
-----
10       95        82
1 rows fetched.
```

- 此处可随堂演示
- create table math (sid int, score int);
- insert into math values(10,95),(11,87),(12,99);
- create table english (sid int, score int);
- insert into english values(10,82),(11,87),(12,93);

别名

- 通过使用子句AS some_name，可以为表名称或列名称指定另一个标题名显示，一般创建别名是为了让列名称的可读性更强。
- 语法格式
 - 列和表的SQL别名分别跟在相应的列名和表名后面，中间可以加或不加一个“AS”关键字。请参见下方示例：
- 示例：别名使用。

```
SELECT staff_id AS empno,course_name FROM training;
```

EMPNO	COURSE_NAME
10	SQL majorization
10	information safety
10	master all kinds of thinking methonds.

```
select a.sid, a.score math, b.score english from math a, english b where a.sid = 10 and b.sid = 10;
```

SID	MATH	ENGLISH
10	95	82

- SELECT staff_id "empno" ,course_name FROM training; 通过加双引号的方式也可以来表示别名

条件查询 (1)

- 在SELECT语句中，可以通过设置条件以达到更精确的查询。条件由表达式与操作符共同指定，且条件返回的值是TRUE，FALSE或UNKNOWN。查询条件可以应用于WHERE子句，HAVING子句。

- 语法格式：

- condition 子句

```
select_statement { predicate } [ { AND | OR } condition ] [ , ... n ]
```

- predicate 子句

```
{ expression { = | <> | != | > | >= | < | <= } { ALL | ANY } expression | ( select )  
| string_expression [ NOT ] LIKE string_expression  
| expression [ NOT ] BETWEEN expression AND expression  
| expression IS [ NOT ] NULL  
| expression [ NOT ] IN ( select | expression [ , ... n ] )  
| [ NOT ] EXISTS ( select )  
}
```

条件查询 (2)

- 查询条件由表达式和操作符共同定义。常用的条件定义方式如下：
 - 比较操作符 “>”, “<”, “>=”, “<=”, “!=”, “<>”, “=”指定的比较查询条件。当查询条件中和数字比较, 可以使用单引号引起, 也可以不用, 当和字符及日期类型的数据比较, 则必须用单引号引起。
 - 测试运算符指定的范围查询条件。如果希望返回的结果必须满足多个条件, 可以使用AND逻辑操作符连接这些条件; 如果希望返回的结果满足多个条件之一即可, 可以使用OR逻辑操作符连接这些条件。
- 示例: 使用比较操作符来指定查询条件, 例如查询学习课程SQL majorization的人员信息。

```
SELECT * FROM training WHERE course_name = 'SQL majorization';
```

STAFF_ID	COURSE_NAME	EXAM_DATE	SCORE
10	SQL majorization	2017-06-25 12:00:00	90

1 rows fetched.

条件查询 (3)

- 逻辑操作符

- 常用的逻辑操作符有AND、OR和NOT，他们的运算结果有三个值，分别为TRUE、FALSE和NULL，其中NULL代表未知。他们运算优先级顺序为：NOT>AND>OR。

- 运算规则表

a	b	a AND b	a OR b	NOT a
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL

条件查询 (4)

- GaussDB(for openGauss)支持如下表的测试运算符：

运算符	描述
IN/NOT IN	元素在/不在指定的集合中。
EXISTS/NOT EXISTS	存在/不存在符合条件的元素。
ANY/SOME	存在一个值满足条件。SOME是ANY的同义词。
ALL	全部值满足条件。
BETWEEN...AND...	在两者之间，例如a BETWEEN x AND y等效于a>= x and a <= y。
IS NULL/IS NOT NULL	等于/不等于NULL。
LIKE/NOT LIKE	字符串模式匹配/不匹配。
REGEXP	字符串与正则表达式相匹配，仅支持STRING类型。
REGEXP_LIKE	字符串与正则表达式相匹配，支持STRING类型和NUMBER类型。

条件查询 (5)

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	document developer	7000
37	liming	developer	8000
39	wanghua	tester	8000

- 示例：从上表bonuses_depa中查询岗位为developer，且奖金>8000的职员信息。

```
select * from bonuses_depa where job = 'developer' and bonus > 8000;
STAFF_ID  NAME      JOB      BONUS
-----
30      wangxin  developer  9000
```

- 示例：从上表bonuses_depa中查询姓wang，且奖金在8500~9500之间的职员信息。

```
select * from bonuses_depa where name like 'wang%' and bonus between 8500 and 9500;
STAFF_ID  NAME      JOB      BONUS
-----
30      wangxin  developer  9000
```

- 此处可随堂演示。
- create table bonuses_depa(staff_id int,name varchar(10), job varchar(20),bonus int);
- insert into bonuses_depa values(30,'wangxin','developer',9000),(31,'xufeng','document developer',7000),(37,'liming','developer',8000),(39,'wanghua','tester',8000);
- GaussDB支持以下通配符：
 - %：表示任意数量的字符，包括无字符，用于like和not like语句中。
 - _：下划线，表示确切的一个未知字符，用于like和not like语句中。

join连接查询 (1)

- 实际应用中所需要的数据，经常会需要查询两个或两个以上的表。这种查询两个或两个以上数据表或视图的查询叫做连接查询。连接查询通常建立在存在相互关系的父子表之间。

- 语法格式

```
SELECT [ , ... ] FROM table_reference  
      [LEFT [OUTER] | RIGHT [OUTER] | FULL [OUTER] | INNER]  
      JOIN table_reference  
      [ON { predicate } [ { AND | OR } condition ] [ , ... n ]]
```

- table_reference子句

```
{ table_name [ [AS] alias ]  
  | view_name [ [AS] alias ]  
  | ( select query ) [ [AS] alias ]  
}
```

- 相关概念：
- 笛卡尔乘积运算定义了一个关系，两个集合X和Y的笛卡尔积，又称直积，表示为 $X \times Y$ 。如果X关系有I个元组、N个属性，而Y关系有J个元组、M个属性，则他们的笛卡尔乘积将会有“ $I \times J$ ”个元组、“ $N+M$ ”个属性。两个关系中可能有同名属性。
- 例如：AB两表做笛卡尔乘积运算。
- 对A（4条记录）和B（5条记录）两个表做不指定条件的连接查询时，会得到20（ 4×5 ）条记录。
- `SELECT * FROM A, B;`
- table_reference子句中可以是普通表、视图、子查询

join连接查询 (2)

- 当查询的FROM子句中出现多个表时，数据库就会执行连接。

- 查询的SELECT列表可以是这些表中任意一些列。

```
SELECT table1.column, table2.column FROM table1, table2;
```

- 大多数连接查询包含至少一个连接条件，连接条件可以在FROM子句中也可以在WHERE子句中。

```
SELECT table1.column, table2.column FROM table1 JOIN table2 ON(table1.column1 = table2.column2);  
SELECT table1.column, table2.column FROM table1, table2 WHERE table1.column1 = table2.column2;
```

内连接

- 内连接：内连接的关键字为inner join，其中inner可以省略。使用内连接，连接执行顺序必然遵循语句中所写的表的顺序。
- 示例：查询员工ID、最高学历和考试分数。使用training和education两个相关的列（staff_id）做查询操作。

```
select * from training;
STAFF_ID  COURSE_NAME          EXAM_DATE          SCORE
-----
10      SQL majorization      2017-06-25 12:00:00    90
11      BIG DATA             2018-06-25 12:00:00    92
12      Performance Turning   2018-06-29 12:00:00    95
```

```
select * from education;
STAFF_ID  HIGEST_DEGREE GRADUATE_SCHOOL          EDUCATION_NOTE
-----
University 211&985
12      doctor      Peking University          211&985
13      scholar     Peking University          211&985
```

```
SELECT e.staff_id, e.highest_degree, t.score FROM education e JOIN training t ON (e.staff_id = t.staff_id);
STAFF_ID  HIGEST_DEGREE SCORE
-----
11      master      92
12      doctor      95
```

- 此处可以随堂演示
- --删除表training。
- DROP TABLE IF training;
- --创建表training。
- CREATE TABLE training(staff_id INT NOT NULL, course_name CHAR(50), exam_date DATETIME, score INT);
- --向表training中插入记录1。
- INSERT INTO training(staff_id, course_name, exam_date, score) VALUES(10, 'SQL majorization', '2017-06-25 12:00:00', 90);
- --向表training中插入记录2。
- INSERT INTO training(staff_id, course_name, exam_date, score) VALUES(11, 'BIG DATA', '2018-06-25 12:00:00', 92);
- --向表training中插入记录3。
- INSERT INTO training(staff_id, course_name, exam_date, score) VALUES(12, 'Performance Turning', '2018-06-29 12:00:00', 95);
- --删除表education。

外连接

- 内连接所指定的两个数据源处于平等的地位。而外连接不同，外连接以一个数据源为基础，将另外一个数据源与之进行条件匹配。
- 内连接返回两个表中所有满足连接条件的数据记录。外连接不仅返回满足连接条件的记录，还将返回不满足连接条件的记录。
- 外连接又分为左外连接、右外连接和全外连接。

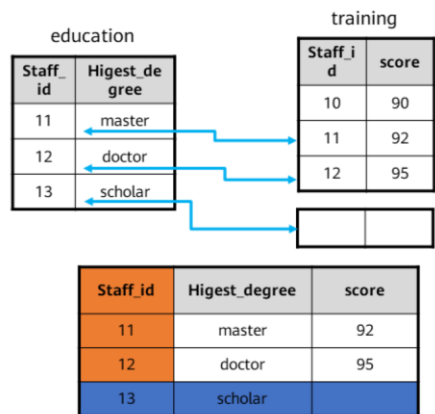
左外连接

- 又称左连接，如图所示，是指以左边的表为基础表进行查询。
- 根据指定连接条件关联右表，获取基础表以及和条件匹配的右表数据，未匹配条件的右表对应的字段位置填上NULL。
- 示例：

```
SELECT e.staff_id, e.higest_degree, t.score FROM education e LEFT JOIN  
training t ON (e.staff_id = t.staff_id);
```

STAFF_ID	HIGEST_DEGREE	SCORE
11	master	92
12	doctor	95
13	scholar	

3 rows fetched.

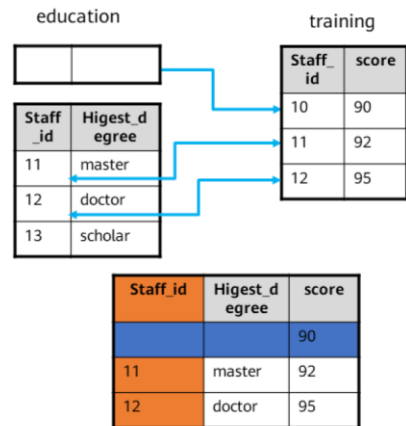


右外连接

- 又称右连接，是指以右边的表为基础表，在内连接的基础上也查询右边表中有记录，而左边的表中没有记录的数据（左边用 NULL 值填充）。
- 示例：

```
SELECT e.staff_id, e.highest_degree, t.score FROM education e RIGHT JOIN training t ON (e.staff_id = t.staff_id);
```

STAFF_ID	HIGHEST_DEGREE	SCORE
		90
11	master	92
12	doctor	95

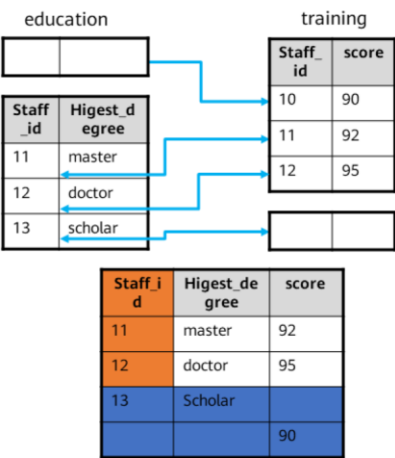


全外连接

- 又称全连接，是指除了返回两个表中满足连接条件的记录，还会返回两个表中不满足连接条件的所有其它行（不匹配的另外一边用NULL值填充）。
- 示例：

```
SELECT e.staff_id, e.higest_degree, t.score FROM education e FULL JOIN training t ON (e.staff_id = t.staff_id);
```

STAFF_ID	HIGEST_DEGREE	SCORE
11	master	92
12	doctor	95
13	scholar	90



半连接

- 半连接（Semi Join）是一种特殊的连接类型，在SQL中没有指定的关键字，通过在WHERE后面使用IN或EXISTS子查询实现。当IN/EXISTS右侧的多行满足子查询的条件时，主查询也只返回一行与IN/EXISTS子查询匹配的行，而不是复制左侧的行。
- 示例：查看参加培训的员工的教育信息。即使training表中的许多行可能与子查询匹配（即同一个staff_id下有多个职员），也只需要从表training返回一行。

```
SELECT staff_id, highest_degree, education_note FROM education WHERE EXISTS (SELECT * FROM training WHERE education.staff_id = training.staff_id);
```

STAFF_ID	HIGEST_DEGREE	EDUCATION_NOTE
----------	---------------	----------------

11	master	211&985
12	doctor	211&985

反连接

- 反连接（ Anti Join ）是一种特殊的连接类型，在SQL中没有指定的关键字，通过在WHERE后面使用NOT IN或NOT EXISTS子查询实现。返回所有不满足条件的行。这个关系的概念跟半连接相反。
- 示例：查询没有参加培训的员工的教育信息。

```
SELECT staff_id, highest_degree, education_note FROM education WHERE staff_id NOT IN (SELECT staff_id FROM training);
```

STAFF_ID	HIGEST_DEGREE	EDUCATION_NOTE
----------	---------------	----------------

13	scholar	211&985
----	---------	---------

子查询 (1)

- 子查询是指在查询、建表或插入语句的内部嵌入查询，以获得临时结果集。
 - 子查询可以分为相关子查询和非相关子查询；
 - 子查询的语法格式与普通查询相同。
- 使用方法
 - 子查询可以出现在FROM子句、WHERE子句、以及WITH AS子句中。FROM子句中的子查询也称为内联视图。WHERE子句中的子查询也称为嵌套子查询。

- 相关子查询：执行查询的时候先取得外层查询的一个属性值，然后执行与此属性值相关的子查询，执行完毕后再取得外层查询的下一个值，依次再来重复执行子查询。即：
 - 在子查询中引用了外部查询表中的列。
 - 子查询的值依赖于外部查询表中的列的值。
 - 对于外部查询中的每一行，子查询都要执行一次。
- 非相关子查询：子查询独立于外层语句（主查询），子查询的执行不需要提前取得父查询的值，只是作为父查询的查询条件。查询执行时子查询和主查询可分为两个独立的步骤，即先执行子查询，再执行主查询。

子查询 (2)

- WITH AS子句
- 定义一个SQL片断，该SQL片断会被整个SQL语句用到。

- 语法格式

```
WITH { table_name AS select_statement1 }[, ...] select_statement2
```

- table_name
- 用户自定义的存储SQL片段的表的名称。
- select_statement1
- 从基本表中查询数据的SELECT语句。
- select_statement2
- 从用户自定义的存储SQL片段的表中查询数据的SELECT语句。

- 可以使SQL语句的可读性更高。存储SQL片段的表与基本表不同，是一个虚表。数据库不存放视图对应的定义和数据，这些数据仍存放在原来的基本表中。若基本表中的数据发生变化，从存储SQL片段的表中查询出的数据也随之改变。

子查询示例 (1)

- 通过相关子查询，查找每个部门中高出部门平均工资的人员。

```
SELECT s1.last_name, s1.section_id, s1.salary
FROM staffs s1
WHERE salary > (SELECT avg(salary) FROM staffs s2 WHERE s2.section_id = s1.section_id)
ORDER BY s1.section_id;
```

- 对于staffs表的每一行，父查询使用相关子查询来计算同一部门成员的平均工资。相关子查询为staffs表的每一行执行以下步骤：
 - 确定行的section_id。
 - 然后使用section_id来评估父查询。
 - 如果此行中工资大于所在部门的平均工资，则返回该行。
- 对于staffs表的每一行，子查询都将被计算一次。

子查询示例 (2)

- WITH子查询：查询培训过big data课程的员工信息。

```
WITH bigdata_staffs AS (select staff_id,exam_date from training where course_name = 'BIG DATA' )select * from bigdata_staffs;
```

STAFF_ID	EXAM_DATE
11	2018-06-25 12:00:00

- 通过子查询建立一个和表training具有相同表结构的表。

```
CREATE TABLE training_new AS SELECT * FROM training WHERE 1<>1;
```

- 通过子查询向表training_new表中插入training表的所有数据。

```
INSERT into training_new SELECT * FROM training;
```

- 此处可随堂演示
- <>是不等于的意思，1<>1的条件不成立，所以子查询不会返回数据。

合并结果集 (1)

- 除子查询外，还可以使用集合运算符处理多个查询的结果集，输出最终结果。

- Union运算符：将多个查询块的结果集合并为一个结果集输出。

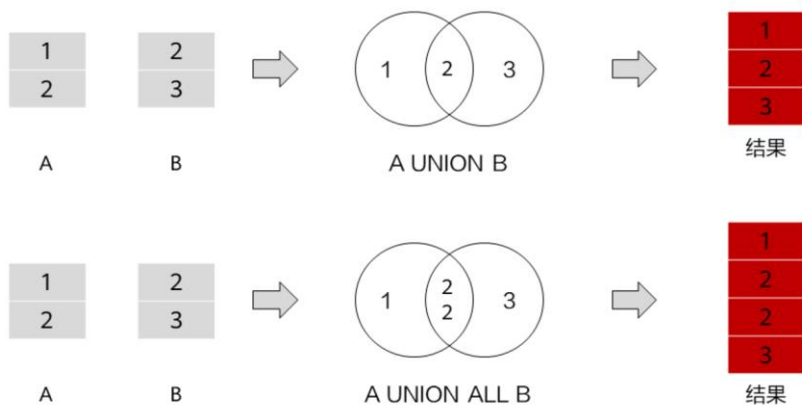
```
select_statement UNION [ALL] select_subquery
```

- 使用方法

- 每个查询块的查询列数目必须相同。
- 每个查询块对应的查询列必须为相同数据类型或同一数据类型组。
- 关键字ALL的意思是保持所有重复数据，而没有ALL的情况下表示删除所有重复数据。

合并结果集 (2)

- 图解



- UNION去重，UNION ALL不去重

合并结果集 (3)

- 示例：现有两个部门的员工信息，查询获得奖金超过7000的员工信息。

bonuses_depa1

staff_id	name	job	bonus
23	wangxia	Developer	5000
24	limingying	Tester	7000
25	liulili	quality control	8000
29	liuxue	tester	9000

bonuses_depa2

staff_id	name	job	bonus
30	wangxin	developer	9000
31	xufeng	Document developer	6000
34	denggui	quality control	5000
35	caoming	tester	10000

```
SELECT staff_id, name, bonus FROM bonuses_depa1 WHERE bonus > 7000 UNION ALL SELECT
staff_id, name, bonus FROM bonuses_depa2 WHERE bonus > 7000 ;
```

STAFF_ID	STAFF_NAME	BONUS
30	wangxin	9000
35	caoming	10000
25	liulili	8000
29	liuxue	9000

其他常用查询操作

- GaussDB(for openGauss)还支持：

- 合并结果集(UNION [ALL])
- 差异结果集(MINUS/EXCEPT)
- 数据分组操作(GROUP BY)
- 数据排序操作(ORDER BY)
- 数据限制(LIMIT)

- union与union all的差异：

- union会将两个结果集进行合并，并进行去重。之后将结果进行排序。
- union all只是单纯地将两个集合合并。

- minus是用第一个集合减去第二个集合，保留第一个集合中和第二个集合不相同的记录。例如，集合A：{1,2}。集合B：{2,3}，则A minus B 结果为 1。

思考题

1. （单选题）查询集合操作中，表示合并结果集的是？（ ）

- A. INTERSECT
- B. MINUS
- C. EXCEPT
- D. UNION

· 参考答案： D



目录

1. 数据类型
2. 常用数据库对象
3. 函数与操作符
4. 数据查询
- 5. 数据操纵**

数据操纵语言

- DML（Data Manipulation Language数据操纵语言），用于对数据库表中的数据进行操作。如：插入、更新、删除等。
- 插入数据:
 - 使用INSERT命令往数据库表中添加一条或多条记录。
- 删除数据:
 - 使用DELETE删除表中指定的数据，或用TRUNCATE删除表的所有数据。
- 修改数据:
 - 使用UPDATE修改数据是修改数据库表中的一条或多条记录。
- 拷贝数据:
 - 使用COPY拷贝表和数据文件之间的数据。

数据插入

- 功能描述

- 在表中插入新的数据。

- 注意事项

- 只有拥有表INSERT权限的用户，才可以向表中插入数据。
- 如果使用RETURNING子句，用户必须要有该表的SELECT权限。
- 如果使用query子句插入来自查询里的数据行，用户还需要拥有在查询里使用的表的SELECT权限。

- 注意事项

- 执行该语句的用户需要有表的INSERT权限或者INSERT ANY TABLE的系统权限。普通用户不允许insert系统SYS用户对象。
- INSERT ...SELECT形式，select_list列数必须与待插入的字段数一样。
- INSERT中单行数据量需小于64000字节。

语法格式

- INSERT语句有三种形式。

- 查询插入，通过VALUES子句构造一行或多行记录插入到表中。

```
INSERT [hint_info] [IGNORE] [ INTO ] [ schema_name. ]table_name [ ( column_name  
[ , ... ] ) ] VALUES ( expression [ , ... ] )
```

- 查询插入，通过SELECT子句返回的结果集构造一行或多行记录插入到表中。

```
INSERT [IGNORE] [ INTO ] [ schema_name. ]table_name [table_alias] [ ( column_name  
[ , ... ] ) ]select_clause
```

- 先插入记录，如果报主键冲突错误则执行UPDATE操作，更新指定字段值。

```
INSERT [ INTO ] [ schema_name. ]table_name [ ( column_name [ , ... ] ) ] VALUES  
( expression [ , ... ] ) ON DUPLICATE KEY UPDATE {column_name = expression} [ , ... ]
```

- 参数说明

- IGNORE
 - 用于忽略会导致重复关键字错误的记录，不支持和ON DUPLICATE KEY UPDATE 同时使用。
- table_name
 - 待插入的表名。
- column_name
 - 待插入的表字段名。
 - 如果insert语句所指定的字段名包含表中的所有字段，则可能省略字段名。
 - 取值范围：已存在的字段名。
- expression
 - 插入字段的值或表达式。
- select_clause
 - SELECT查询结果集作为新值插入到表中，具体参数，参考select章节。
- select_list
 - 指定查询列。

数据插入示例

- 示例：向表training1中插入数据。

- 创建表training1。

```
CREATE TABLE training1(staff_id INT NOT NULL, course_name CHAR(50), exam_date DATETIME, score INT);
```

- 值插入：向表training1中插入一条记录。

```
INSERT INTO training1(staff_id, course_name, exam_date, score) VALUES(1, 'information safety', '2017-06-26 12:00:00', 95);
```

- 查询插入：通过子查询向表training1表中插入training表的所有数据。

```
INSERT INTO training1 SELECT * FROM training;
```

- 主键冲突错误，执行UPDATE操作。

```
--创建主键
ALTER TABLE training1 ADD PRIMARY KEY (staff_id);
--插入记录
INSERT INTO training1 VALUES (1, 'master all kinds of thinking methonds', '2017-07-25 12:00:00', 97) ON DUPLICATE KEY UPDATE course_name = 'master all kinds of thinking methonds', exam_date = '2017-07-25 12:00:00', score = 97;
```

- 前面已经创建了training表。
 - 主键是一个在一个数据库表中唯一标识每个行/记录表中的一个字段。主键不能为NULL值且必须包含唯一值。
 - 主键冲突错误，执行UPDATE操作。这里由于training1表中主键staff_id已经存在值1，所以执行update操作

数据修改

- 功能描述
 - 更新表中行的值。
- 注意事项：
 - 不支持UPDATE语句中直接使用LIMIT，应使用WHERE条件明确需要更新的目标行。
 - 不支持多表更新。多表更新即在单条SQL语句中，对多个表进行更新。
 - UPDATE语句中必须有WHERE子句，避免全表扫描。
 - UPDATE语句中禁止使用ORDER BY、GROUP BY子句，避免不必要的排序。

语法格式

```
UPDATE [ ONLY ] table_name [ * ] [ [ AS ] alias ]
SET {column_name = { expression | DEFAULT }
| ( column_name [, ...] ) = {( { expression | DEFAULT } [, ...] ) |sub_query }}[, ...]
[ FROM from_list] [ WHERE condition ]
[ RETURNING { *
| {output_expression [ [ AS ] output_name ]} [, ...] }];
```

- 参数说明：

- table_name：要更新的表名
- column_name：要修改的字段名
- sub_query：使用子查询的方法来更新这个表
- Condition：一个返回boolean类型的表达式，只有在这个表达式返回true的行才会被更新。

- 参数说明
- table_reference
- 要更新的表、表集合。取值范围：已存在的表、表集合。
- table_name
- 要更新的表名。取值范围：已存在的表名称。
- col_name
- 要修改的字段名。取值范围：已存在的字段名。
- expression
- 赋给字段的值或表达式。
- condition
- 一个返回布尔类型结果的表达式。只有这个表达式返回true的行才会被更新。
- join_table
- 用于关联查询的一组表集合。
- [INNER] JOIN 用于取两表的交集。
- LEFT [OUTER] JOIN 用于取左表的全集，右表不匹配的以null值代替。
- RIGHT [OUTER] JOIN 用于取右表的全集，左表不匹配的以null值代替。

数据修改示例

```
-创建表student1。
postgres=# CREATE TABLE student1(stuno int, classno int);
--插入数据。
postgres=# INSERT INTO student1 VALUES(1,1);
postgres=# INSERT INTO student1 VALUES(2,2);
postgres=# INSERT INTO student1 VALUES(3,3);
--查看数据。
postgres=# SELECT * FROM student1;
--直接更新所有记录的值。
postgres=# UPDATE student1 SET classno = classno*2;
--查看数据。
postgres=# SELECT * FROM student1;
--删除表。
postgres=# DROP TABLE student1;
```

数据删除

- 功能描述
 - 从表中删除行。
- 注意事项
 - 不支持DELETE语句中使用LIMIT。应使用WHERE条件明确需要更新的目标行。
 - 不支持多表删除。多表删除即在单条SQL语句中，对多个表进行删除。
 - 如果需要清空一张表，建议使用TRUNCATE，而不是DELETE。TRUNCATE会创建新的物理文件，并在事务结束时将原文件物理删除，清空磁盘空间。而DELETE会将表中数据进行标记，直到VACCUUM FULL阶段才会真正清理磁盘空间。

语法格式

```
DELETE FROM [ ONLY ] table_name [ * ] [ [ AS ] alias ]
[ USING using_list ]
[ WHERE condition | WHERE CURRENT OF cursor_name ] [ LIMIT row_count ]
[ RETURNING { * | { output_expr [ [ AS ] output_name ] } [, ...] } ];
```

- 主要参数

- table_name: 目标表的名称
- condition: 一个返回boolean值的表达式，用于判断哪些行需要被删除。

- table_ref_list子句:

- [schema_name.]table_name
- join_table 子句:
- table_reference [LEFT [OUTER] | RIGHT [OUTER] | INNER] JOIN table_reference ON conditional_expr
- table_reference 子句:
- { [schema_name.]table_name [[AS] alias]
- | [schema_name.]view_name [[AS] alias]
- | (select query) [[AS] alias]
- | join_table
- }

- 参数说明

- [schema_name.]table_name
- 要删除数据的表的名称。
- condition
- 指定删除数据要满足的条件。

数据删除示例

- 创建表tpcds.customer_address_bak。

```
CREATE TABLE tpcds.customer_address_bak AS TABLE tpcds.customer_address;
```

- 删除tpcds.customer_address_bak中ca_address_sk小于14888的职员。

```
DELETE FROM tpcds.customer_address_bak WHERE ca_address_sk < 14888;
```

- 删除tpcds.customer_address_bak中所有数据。

```
DELETE FROM tpcds.customer_address_bak;
```

- 删除tpcds.customer_address_bak表。

```
DROP TABLE tpcds.customer_address_bak;
```

思考题

1. (单选题)删除表student中班级(cid)为6的全部学生信息, 下面SQL语句正确的是()?

- A. delete from student where cid = 6;
- B. delete * from student where cid = 6;
- C. delete from student on cid = 6;
- D. Delete * from student on cid = 6;

· 参考答案: A

总结

- 本章主要围绕GaussDB(for openGauss)云数据库中常用的数据类型、数据库对象、系统函数及操作符等内容展开进行介绍，并帮助初学者掌握SQL入门级的基础语法。

缩略语

- SQL : Structured Query Language 结构化查询语言
- DDL : Data Definition Language 数据定义语言
- DML : Data Manipulation Language 数据定义语言



感谢

版权所有©2022，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编，也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为云 + 智能，见未来

www.huaweicloud.com